

Data returning Rules:

The data returned by every routes should be in this format

Normal Response Format

```
{
  status: String,
  length?: Number,
  data: {
    // Data returned by route goes here
  }
}
```

Error format

```
{
  status: String,
  code: Number,
  message: String [ explain the cause in good way ],
  stack?: String [ only if in development mode ]
}
```

Global Rules:

- Every route that is receiving **JWT** in the request authorization header should always verify the token and then proceed forward
 - Should also check if the user has recently changed the password if so then the token is invalid
- Data sanitization and validation should be done
 - Sanitization Includes
 - removing/escaping special symbols that can lead to illegal operations either at the backend or frontend
 - Validation Includes
 - Check if the required data is not empty or not of the required data type
 - Those routes that have the **PATCH** request method can accept empty data in this case the old data in the database will be preserved
 - Check the data range limits as well. For instance, the string should be restricted to a certain range, and the file size shouldn't be greater than the limit.
 - Before doing any operation that modifies something verify that the user is valid [ie. owner] and he has the right to do so and also check that the user still exists
- Messages End-to-End Encryption [Optional]
 - If the no of members are greater than 2 then Tree-based Group Diffie-Hellman (TGDH) encryption algorithm will be used to encrypt messages
 - If no of members are 2 then Advanced Encryption Standard (AES) algorithm will be used to encrypt messages
 - To share secret keys between parties Elliptic Curve Diffie-Hellman (ECDH) will be used

Routes Restriction Rules

- **Admin has access to**
 - /users
 - /users/:id
- **Talent has access to**
 - Access to all routes except admin-restricted ones
- **Client has access to**
 - Access to all routes except admin-restricted ones

Authentication Service End-Points

Path	Method	Body?	Return	Goal?
/login	POST	{ email, password }	{ JWT }	Validate email & password and return data
/signup	POST	{ name, email, password, userType, phoneNo, image }	{ JWT }	Create user and return data
/currentUser	GET	-	{ id, name, email, phoneNo, userType, image, user_status, verified, financeAllowed, blocked }	Return the current user
/users	GET	-	{ id, name, email, phoneNo, userType, image, user_status, verified, financeAllowed, blocked }	Return the all the users
/users/:id	GET	-	{ id, name, email, phoneNo, userType, image, user_status, verified, financeAllowed, blocked }	Return the user with matching id
/users/:id	PATCH	{ JWT, name, email, password, phoneNo, image }	{ id, name, email, phoneNo, userType, image, user_status, verified, financeAllowed, blocked }	Update user info and return data
/users/:id	DELETE	-	-	Mark the user as inactive with matching id
/resetpassword/:resettokken	POST	{ email, newPassword }	-	Reset user password
/forgetpassword	POST	{ email }	-	Generate reset token and mail the rest URL to the user

Authentication Service Events

Channel	Data	Description
user:created	{ user [included all attributes] }	This event will be published when a user is created with given data
user:updated	{ user [included all attributes] }	This event will be published when the user is updated with given data
user:deleted	{ id }	This event will be published when a user is delete with the given data
user:forgetPassword	{ uuid, resetToken, resetTokenExpires }	This event will be published when the user request to change its password
user:resetPassword	{ uuid, newPassword, passwordChangedAt }	This event will be published when the user password is reseted
Queue Group Name	auth-service-queue-group	

User Collection	
PK	<u>Id</u>
	name: String { required, not null } email: String { required, not null } password: String { required, not null } passwordChangedAt: Date { default: Now } resetToken: String { default: null } resetTokenGeneratedAt: Date { default: null } otp: Number [min: 4 Digit, max: 4 Digit] { default: null } otpGeneratedAt: Date { default: null } isActive: boolean { default: true } invalidLoginCount: Number [max count 5] { default: 0 } userType: Array ['freelancer', 'client', 'Admin'] { default: 'client', required, not null } phoneNo: String { required } image: Object { id, url, default: null } userStatus: String ['Online', 'Offline'] { default: 'Online' } verified: boolean { default: false } joinedDate: Date { default: Now } financeAllowed: boolean { default: true } blocked: Array [isBlocked: boolean { default: false }, reason: String { default: null }]

Database Dependencies	
Microservice Name	Table / Collection Name
None	

Routes Restriction Rules

- **Admin has access to**
 - /profiles/users/:id [Method: GET, PATCH, DELETE]
 - /profiles/:id/projects [Method: GET]
 - /profiles/:id/projects/:projectId [Method: GET, PATCH, DELETE]
 - /profiles/:id/projects/:projectId/attachments [Method: GET]
 - /profiles/:id/projects/:projectId/attachments/:attachmentId [Method: DELETE]
- **Talent has access to**
 - All routes
 - Can't PATCH, DELETE anything from other users profiles
- **Client has access to**
 - /profiles/users/:id [Method: GET]
 - /profiles/:id/projects [Method: GET]
 - /profiles/:id/projects/:projectId [Method: GET]

Points to be noted:

- /profile { Method: POST } will not be implemented as when the user is created then the auth service will fire an event it is the responsibility of this service to capture that event and automatically create the profile of that user

Profile Service End-Points

Path	Method	Body?	Return	Goal?
/profiles/users/:id	GET	-	{ id, about, languages, dob, gender, ranking, address, location, skills }	Return profile of matching user id
/profiles/users/:id	PATCH	{ about, languages, dob, gender, ranking, address, location, skills }	{ id, about, languages, dob, gender, ranking, address, location, skills }	Update the profile and return updated
/profiles/users/:id	DELETE	-	-	Mark the user profile as inactive
/profiles/projects	POST	{ title, description, startDate, endDate, attachments: { mimeType, file }, skills, contractId }	{ id, title, description, startDate, endDate, attachments: { id, mimeType, orignalName, url, createdAt }, skills, feedback }	Create a project for the current user
/profiles/:id /projects	GET	-	{ id, title, description, startDate, endDate, attachments: { id, mimeType, orignalName, url, createdAt }, skills, feedback }	Return projects of matching profile id
/profiles/:id /projects/:projectId	GET	-	{ id, title, description, startDate, endDate, attachments: { id, mimeType, orignalName, url, createdAt }, skills, feedback }	Return project with associated id
/profiles/:id /projects/:projectId	DELETE	-	-	Delete the project with the matching id
/profiles/:id /projects/:projectId	PATCH	{ title, description, startDate, endDate, skills, contractId }	{ id, title, description, startDate, endDate, attachments: { id, mimeType, orignalName, url, createdAt }, skills, feedback }	Update project of matching id
/profiles/:id/projects /:projectId /attachments	POST	{ attachments: { mimeType, file } }	attachments: { id, mimeType, orignalName, url, createdAt }	Append attachments to the project of matching id
/profiles/:id/projects /:projectId /attachments	GET	-	attachments: { id, mimeType, orignalName, url, createdAt }	Return attachments to the project of matching id
/profiles/:id /projects/:projectId /attachments /:attachmentid	GET	-	attachment: { id, mimeType, orignalName, url, createdAt }	Return attachments of matching id to the project of matching id
/profiles/:id /projects/:projectId /attachments /:attachmentid	DELETE	-	-	Delete the attachment with the matching id
/profiles/:id /projects/:projectId /attachments	DELETE	-	-	Delete all attachment with the matching project id
/profiles/:id/projects /:projectId/skills	POST	{ skills: [skillId] }	-	Append skills of the project of matching id
/profiles/:id/projects /:projectId/skills	GET	-	{ skills }	Return skills of the project of matching id
/profiles/:id/projects /:projectId/skills /:skillId	DELETE	-	{ skills }	Delete the skill with the matching id and returns skills
/profiles/:id/projects /:projectId/skills	DELETE	{ skills: [id] }	{ project }	Delete all the skills with the matching ids
/profiles/:id/projects /:projectId/skills	DELETE	-	{ project }	Delete all the skills with the matching project id

Profile Service Events

Channel	Data	Description
---------	------	-------------

Channel	Data	Description
---------	------	-------------

Channel	Data	Description
---------	------	-------------

No events will be fired		
-------------------------	--	--

Profiles Collection	
PK	<u>Id</u>
	about: String languages: Array [{ name, experience }] { required, not null } dob: Date { required, not null } ranking: String ['rising_talent', 'top Rated', 'top Rated Plus'] { default: null } address: String { default: null } location: Date { required, not null } projects: Array [id] { default: null } skills: Array [id] { default: null } user_id: ObjectID()

Projects Collection	
PK	<u>id</u>
	title: String { required, not null } description: String [max-character: 350] { required, not null } start_date: Date { required, default: Date.now() } end_date: Date { required, not null } attachments: Array [{ id, mimeType, originalName, url, createdAt }] skills: Array [id: ObjectID()] contractId: ObjectID()

Database Dependencies	
Microservice Name	Table / Collection Name
Authentication	User
Catalog	Skills

Routes Restriction Rules • Admin has access to ◦ /catelogs/products/users/:id [Method: GET] ◦ /catelogs/products/:id [Method: GET, DELETE] ◦ /catelogs/products/:id/attachments [Method: GET, DELETE] ◦ /catelogs/products/:id/attachments/:attachmentId [Method: GET, DELETE] ◦ /catelogs/services/users/:id [Method: GET] ◦ /catelogs/services/:id [Method: GET, DELETE] ◦ /catelogs/services/:id/attachments [Method: GET, DELETE] ◦ /catelogs/services/:id/attachments/:attachmentId [Method: GET, DELETE] ◦ /catelogs/orders/user/:id [Method: GET] ◦ /catelogs/orders/:id/users/:userId [Method: GET, PATCH, DELETE] • Talent has access to ◦ All routes ◦ Can't PATCH, DELETE anything from other users catalog • Client has access to ◦ /catelogs/orders [Method: POST] ◦ /catelogs/orders/user/:id [Method: GET] ◦ /catelogs/orders/:id/users/:userId [Method: GET, PATCH, DELETE] ◦ /catelogs/products/users/:id [Method: GET] ◦ /catelogs/products/:id [Method: GET] ◦ /catelogs/products/:id/attachments [Method: GET] ◦ /catelogs/products/:id/attachments/:attachmentId [Method: GET] ◦ /catelogs/services/users/:id [Method: GET] ◦ /catelogs/services/:id/attachments [Method: GET] ◦ /catelogs/services/:id/attachments/:attachmentid [Method: GET]				
--	--	--	--	--

Catalog Service End-Points				
Path	Method	Body?	Return	Goal?
/catelogs /products	POST	{ title, description, attachments: { mimeType, file }, quantity, ammount }	{ id, title, description, attachments: { id, mimeType, orignalName, url, createdAt }, quantity, ammount, active}	Create product and return the data
/catelogs /products/users/:id	GET	-	{ id, title, description, attachments: { id, mimeType, orignalName, url, createdAt }, quantity, ammount, active }	Return products created by the user
/catelogs /products/:id	GET	-	{ id, title, description, attachments: { id, mimeType, orignalName, url, createdAt }, quantity, ammount, active}	Return the product with associated id
/catelogs /products/:id	DELETE	-	-	Delete the product with the provided id of the current user
/catelogs /products/:id	PATCH	{ title, description, quantity, ammount, active }	{ id, title, description, attachments: { id, mimeType, orignalName, url, createdAt }, quantity, ammount, active}	Update the product with associated id
/catelogs/products /:id/attachments	POST	{ attachments: { mimeType, file } }	attachments: { id, mimeType, orignalName, url, createdAt }	Append attachment to the product of matching id
/catelogs/products /:id/attachments	GET	-	attachments: { id, mimeType, orignalName, url, createdAt }	Return attachment to the product of matching id
/catelogs/products /:id/attachments /:attachmentId	GET	-	attachment: { id, mimeType, orignalName, url, createdAt }	Return the attachment of the matching id to the product of the matching id
/catelogs/products /:id/attachments /:attachmentId	DELETE	-	-	Delete the attachment of the matching id to the product of the matching id

/catelogs/products/:id/attachments	DELETE	-	-	Delete the attachments to the product of the matching id
/catelogs/services	POST	{ name, price, unit [hour, day], images, videos, active }	{ id, name, price, unit [hour, day], images, videos, active, feedback }	Create service and return the created service
/catelogs/services/users/:id	GET	-	{ id, name, price, unit [hour, day], images, videos, active, feedback }	Return the service of the user of matching id
/catelogs/services/:id	GET	-	{ id, name, price, unit [hour, day], images, videos, active, feedback }	Return service with associated id
/catelogs/services/:id	POST	{ message, price, unit }	{ roomId, contractId }	Create a proposal and initiate a contract and send the message to the seller of the service
/catelogs/services/:id	DELETE	-	-	Verify JWT and delete the service with the provided id of the current user
/catelogs/services/:id	PATCH	{ name, price, unit [hour, day], active, feedback }	{ id, name, price, unit [hour, day], images, videos, active, feedback }	Verify JWT and return user services
/catelogs/services/:id/attachments	POST	{ attachments: { mimeType, file } }	attachments: { id, mimeType, originalName, url, createdAt }	Append attachments to the service of matching id
/catelogs/services/:id/attachments	GET	-	attachments: { id, mimeType, originalName, url, createdAt }	Return attachments to the service of matching id
/catelogs/services/:id/attachments/:attachmentid	GET	-	attachment: { id, mimeType, originalName, url, createdAt }	Return attachments of matching id to the service of matching id
/catelogs/services/:id/attachments/:attachmentid	DELETE	-	-	Delete the attachment with the matching id
/catelogs/services/:id/attachments	DELETE	-	-	Delete all attachment with the matching service id
/catelogs/orders	POST	{ productId, quantity, ammount }	{ id, product }, quantity, ammount, status, createdAt }	Create order and return the data
/catelogs/orders/user/:id	GET	-	{ id, product }, quantity, ammount, status, createdAt }	Return all the orders placed by the user or received by the vendor
/catelogs/orders/:id/users/:userId	GET	-	{ id, product }, quantity, ammount, status, createdAt }	Return an order with a matching id of the user can be talent or the user who placed the order
/catelogs/orders/:id/users/:userId	PATCH	{ quantity, ammount, status }	{ id, product }, quantity, ammount, status, createdAt }	Update the order with a matching id of the user can be talent or the user who placed the order the talent can only update status
/catelogs/orders/:id/users/:userId	DELETE	-	-	Mark the order as canceled

Catalog Service Events		
Channel	Data	Description
product:created	{ product } [with all atributes]	This event will be fired whenever a product will be created
product:updated	{ product } [with all atributes]	This event will be fired whenever the product is updated
product:deleted	{ id }	This event will be fired whenever a product will be deleted
service:created	{ service } [with all atributes]	This event will be fired whenever a service will be created
service:updated	{ service } [with all atributes]	This event will be fired whenever the mentioned fields are updated
service:deleted	{ id }	This event will be fired whenever a service will be deleted
order:created	{ order } [with all atributes]	This event will be fired whenever a order will be created
order:updated	{ order: { id, name, price, unit [hour, day], active, feedback } }	This event will be fired whenever the mentioned fields are updated
order:deleted	{ id }	This event will be fired whenever a order will be deleted
Queue Group Name	catalog-service-queue-group	

Projects Collection	
PK	<u>Id</u>
	title: String { required } description: String { required } startDate: Date { required } endDate: Date { required } attachments: Array [{ id, mimeType, originalName, url, createdAt }] skills: [id] contractId: ObjectId() userId: ObjectId()

Products Collection	
PK	<u>Id</u>
	title: String { required } description: String { required } quantity: Number { required, default: 1 } ammount: Float { required, default: 0.0 } active: boolean { default: true } attachments: Array [{ id, mimeType, originalName, url, createdAt }] userId: ObjectId()

Services Collection	
PK	<u>Id</u>
	title: String { required } price: Float { required, default: 0.0 } unit: String ['hourly', 'fixed'] { required, default: 'fixed' } attachments: Array [{ id, mimeType, originalName, url, createdAt }] feedback: Array [String] userId: ObjectId()

Orders Collection	
PK	<u>Id</u>
	productId: ObjectId() { required } quantity: Number { required, default: 1 } amount: Float { required, default: 0.0 } status: String ['Draft', 'Pending', 'Processing ', 'Completed', 'Cancelled ', 'Refunded'] { required, default: 'Draft' } userId: ObjectId () created: Date { default: Date.now } locked: bloean { default: false }

Database Dependencies

Microservice Name	Table / Collection Name
Authentication	User

Routes Restriction Rules	
• Admin has access to /jobs [Method: GET] /jobs/:id [Method: GET, DELETE] /jobs/:id/feadback [Method: PATCH] /jobs/:id/attachments [Method: GET, DELETE] /jobs/:id/attachments/:attachmentId [Method: GET, DELETE] /proposals/:id [Method: GET, DELETE] /proposals/jobs/:id [Method: GET] /contracts [Method: GET] /contracts/:id [Method: GET] • Talent has access to /jobs/search?someWordsToSearch?offset&limit [Method: GET] /jobs/:id [Method: GET] /jobs/:id/feadback [Method: POST] /jobs/:id/attachments [Method: GET] /jobs/:id/attachments/:attachmentId [Method: GET] /proposals/jobs/:id [Method: POST, GET] /proposals/:id [Method: GET, DELETE] /proposals/:id/attachments [Method: POST, GET, DELETE] /proposals/:id/attachments/:attachmentId [Method: GET, DELETE] • Client has access to /jobs [Method: POST, GET] /jobs/:id [Method: GET, PATCH] /jobs/:id/feadback [Method: POST] /jobs/:id/attachments [Method: POST, GET, DELETE] /jobs/:id/attachments/:attachmentId [Method: GET, DELETE] /proposals/services/:id [Method: POST, GET] /proposals/:id [Method: GET, DELETE] /proposals/jobs/:id [Method: GET] /contracts [Method: POST, GET] /contracts/:id [Method: GET, PATCH, DELETE]	

Job Service End-Points

Path	Method	Body?	Return	Goal?
/jobs	POST	{ title, description, categories, budget, keywords, expactedDuration, paymentType, attachments: { mimeType, file } }	{ id, title, description, categories, budget, status, keywords, expactedDuration,paymentType, attachments: { id, mimeType, orignalName, url, createdAt }, userId }	Create job and return data
/jobs	GET	-	{ id, title, description, categories, budget, status, keywords, expactedDuration,paymentType, attachments: { id, mimeType, orignalName, url, createdAt }, userId }	Return jobs created by the current user
/jobs/search? someWordsToSearch ?offset&limit&location &price	GET	-	{ id, title, description, categories, budget, status, keywords, expactedDuration,paymentType, attachments: { id, mimeType, orignalName, url, createdAt }, userId }	Search jobs, handle pagination parameters
/jobs/:id	GET	-	{ id, title, description, categories, budget, status, keywords, expactedDuration,paymentType, attachments: { id, mimeType, orignalName, url, createdAt }, userId }	Return job with associated id
/jobs/:id	PATCH	{ title, description, categories, budget, status, keywords, expactedDuration, paymentType }	{ id, title, description, categories, budget, status, keywords, expactedDuration,paymentType, attachments: { id, mimeType, orignalName, url, createdAt }, userId }	Update job with associated id
/jobs/:id	DELETE	-	-	Delete job with associated id if the user is admin
/jobs/:id /attachments	POST	{ attachments: { mimeType, file } }	attachments: { id, mimeType, orignalName, url, createdAt }	Append attachment to the job of matching id
/jobs/:id /attachments	GET	-	attachments: { id, mimeType, orignalName, url, createdAt }	Return attachment to the job of matching id
/jobs/:id /attachments /:attachmentId	GET	-	attachment: { id, mimeType, orignalName, url, createdAt }	Return the attachment of the matching id to the job of the matching id
/jobs/:id /attachments /:attachmentId	DELETE	-	-	Delete the attachment of the matching id to the job of the matching id
/jobs/:id /attachments	DELETE	-	-	Delete the attachments to the job of the matching id
/proposals/jobs/:id	POST	{ bidAmmount, coverLetter, proposedDuration, attachments: { mimeType, file } }	{ id, refId, userId, bidAmmount, status, coverLetter, proposedDuration, attachments: { id, mimeType, orignalName,url, createdAt },clientFeedback, talentFeedback, clientRatting, talentRatting,type }	Create a proposal for the job
/proposals/jobs/:id	GET	-	{ id, refId, userId, bidAmmount, status, coverLetter, proposedDuration, attachments: { id, mimeType, orignalName,url, createdAt },clientFeedback, talentFeedback, clientRatting, talentRatting,type }	Return proposals made against current job if the user is the owner and admin and if the user is talent then return his proposal.
/proposals /services/:id	POST	{ bidAmmount, coverLetter, proposedDuration, attachments: { mimeType, file } }	{ id, refId, userId, bidAmmount, status, coverLetter, proposedDuration, attachments: { id, mimeType, orignalName,url, createdAt },clientFeedback, talentFeedback, clientRatting, talentRatting,type }	Create a proposal for the service means the client wants to buy the service offered by the talent

/proposals /services/:id	GET	-	{ id, refId, userId, bidAmmount, status, coverLetter, proposedDuration, attachments: { id, mimeType, orignalName,url, createdAt },clientFeedback, talentFeedback, clientRatting, talentRatting,type }	Return proposals made against current job if the user is the owner and admin and if the user is talent then return his proposal.
/proposals/:id	GET	-	{ id, refId, userId, bidAmmount, status, coverLetter, proposedDuration, attachments: { id, mimeType, orignalName,url, createdAt },clientFeedback, talentFeedback, clientRatting, talentRatting,type }	Return proposal with associated id
/proposals/:id	DELETE	-	-	Withdraw proposal of the current user and if the user if the owner of the job then reject the proposal
/proposals/:id /interviews	POST	-	{ message }	Mark the proposal status as Hired
/proposals/:id /jobs/i:jobid	POST	-	{ message }	Mark the proposal status as Accepted
/proposal/:id	POST	{ rating, feedback }	{ proposal }	Find the proposal with the matching ID and the check proposal status should be "Hired" Also verify the user who created the job or submitted the proposal before updating the feedback
/proposals/:id /attachments	POST	{ attachments: { mimeType, file } }	attachments: { id, mimeType, orignalName, url, createdAt }	Append attachment to the proposal of matching id
/proposals/:id /attachments	GET	-	attachments: { id, mimeType, orignalName, url, createdAt }	Return attachment to the proposal of matching id
/proposals/:id /attachments /:attachmentId	GET	-	attachment: { id, mimeType, orignalName, url, createdAt }	Return the attachment of the matching id to the proposal of the matching id
/proposals/:id /attachments /:attachmentId	DELETE	-	-	Delete the attachment of the matching id to the proposal of the matching id
/proposals/:id /attachments	DELETE	-	-	Delete the attachments to the proposal of the matching id
/contracts	POST	{ proposalId, ammount }	{ id, proposalId, startTime, endTime, ammount, status }	Create contract and return data
/contracts	GET	-	{ id, proposalId, startTime, endTime, ammount, status }	Return all contracts only if the user is the owner i.e [client, talent]
/contracts/:id	GET	-	{ id, proposalId, startTime, endTime, ammount, status }	Return contract with matching id only if the user is the owner i.e [client, talent]
/contracts/:id	PATCH	{ ammount, status }	{ id, proposalId, startTime, endTime, ammount, status }	Update the contract both the user can update the status but if the client has canceled the contract before acceptance from talent then the talent can not accept the contract however the amount is only updated by the client
/contracts/:id	DELETE	-	-	Delete contract with the matching id only if the user is the owner and if the user is talent then mark the contract as declined

Job Service Events

Channel	Data	Description
job:created	{ job } [with all atributes]	This event will be fired whenever a job will be created
job:updated	{ job } [with all atributes]	This event will be fired whenever the job is updated
job:deleted	{ id }	This event will be fired whenever a job will be deleted
proposal:created	{ proposal } [with all atributes]	This event will be fired whenever a proposal will be created
proposal:updated	{ proposal } [with all atributes]	This event will be fired whenever the proposal is updated
proposal:deleted	{ id }	This event will be fired whenever a proposal will be deleted
contract:created	{ contract } [with all atributes]	This event will be fired whenever a contract will be created
contract:updated	{ contract } [with all atributes]	This event will be fired whenever the contract is updated
contract:deleted	{ id }	This event will be fired whenever a contract will be deleted

Jobs Collection	
PK	<u>Id</u>
	title: String { required } description: String { required } categories: ObjectId() { required } budget: Float { required } status: ['Draft', 'Open', 'In Review', 'Assigned', 'Completed', 'Canceled', 'Disputed'] { required } keywords: [String] { required } expactedDuration: String ['Less then 1 month', '1 to 3 months', '3 to 6 months', 'More then 6 months'] { required } paymentType: String ['hourly', 'fixed'] { required, default: 'fixed' } videos: [String { url }] { default: null } images: [String { url }] { default: null } userId: ObjectId() { required }

Proposals Collection	
PK	<u>Id</u>
	refId: ObjectId () { required, jobId or serviceId } userId: ObjectId () { required } bidAmmount: Float { required } status: ['Draft', 'Submitted', 'Declined', 'Accepted', 'Hired'] { required, default: 'Submited' } coverLetter: String { required } proposedDuration: String ['Less then 1 month', '1 to 3 months', '3 to 6 months', 'More then 6 months'] { required } videos: [String { url }] { default: null } images: [String { url }] { default: null } clientFeedback: String { default: null } clientRatting: Float [0 to 5] { default: 0.0 } talentFeedback: String { default: null } talentRatting: Float [0 to 5] { default: 0.0 } type: String ['job', 'service'] { required }

Contracts Collection	
PK	<u>Id</u>
	proposalId: ObjectId() { required } startTime: Date { required, default: Date.Now() } endTime: Date { required, default: When the status value is " } ammount: Float { required } status: ['Draft', 'Pending Approval', 'In Progress', 'Delivered', 'Revision', 'Payment Pending', 'Completed', 'Canceled'] { required, default: 'Draft' }

Database Dependencies	
Microservice Name	Table / Collection Name
Authentication	User

Status Field Description For Jobs Collection:

- Draft: Indicates that the job has been created but is not yet finalized or submitted for posting.
- Open: Indicates that the job has been posted and is available for freelancers to apply.
- In Review: Indicates that the job is being reviewed by the platform or the client before it is assigned to a freelancer.
- Assigned: Indicates that the job has been assigned to a freelancer and is currently in progress.
- Completed: Indicates that the job has been successfully completed by the freelancer and has been approved by the client.
- Canceled: Indicates that the job has been canceled or terminated due to some reason, such as a client request, freelancer unavailability, or project scope changes.
- Disputed: Indicates that there is a dispute between the client and the freelancer regarding the job, such as payment or quality issues.

Status Field Description For Proposals Collection:

- Draft: Indicates that the proposal has been created but is not yet finalized or submitted for review.
- Submitted: Indicates that the proposal has been submitted for review to the client who posted the job.
- Declined: Indicates that the proposal has been declined by the client who posted the job.
- Accepted: Indicates that the proposal has been accepted by the client who posted the job.
- Hired: Indicates that the freelancer who submitted the proposal has been hired by the client who posted the job.

Status Field Description For Contracts Collection:

- Draft: Indicates that the contract has been created but is not yet finalized or signed by both parties.
- Pending Approval: Indicates that the contract has been sent to the client for review and approval.
- In Progress: Indicates that work on the project has started and the freelancer is working on the job.
- Delivered: Indicates that the freelancer has completed the work and delivered it to the client.
- Revision: Indicates that the work delivered needs to be revised or modified based on client feedback.
- Payment Pending: Indicates that the client needs to make a payment for the work delivered by the freelancer.
- Completed: Indicates that the project has been completed and both parties have agreed to the work delivered.
- Canceled: Indicates that the contract has been canceled or terminated due to some reason, such as a client request, freelancer unavailability, or project scope changes.

Routes Restriction Rules • Admin has access to ◦ /chats/messages/:id [Method: PATCH, DELETE] ◦ /chats/rooms/:id/messages [Method: POST, GET] ◦ /chats/rooms [Method: POST, GET] ◦ /chats/rooms/:id/members [Method: POST, GET] ◦ /chats/rooms/:id/members/:memberId [Method: GET, DELETE] • Talent has access to ◦ /chats/messages/:id [Method: PATCH, DELETE] ◦ /chats/rooms/:id/messages [Method: POST, GET] ◦ /chats/rooms [Method: GET] ◦ /chats/rooms/:id/members [Method: GET] ◦ /chats/rooms/:id/members/:memberId [Method: GET] • Client has access to ◦ /chats/messages/:id [Method: PATCH, DELETE] ◦ /chats/rooms/:id/messages [Method: POST, GET] ◦ /chats/rooms [Method: POST, GET] ◦ /chats/rooms/:id/members [Method: POST, GET]	
--	--

Chat Service End-Points

Path	Method	Body?	Return	Goal?
/chats/messages/:id	PATCH	{ text }	{ id, text, attachments: { id, mimeType, originalName, url, createdAt }, roomId, senderId, status, createdAt }	check if message is not deleted and return message
/chats/messages/:id	DELETE	-	{ id, text, attachments: { id, mimeType, originalName, url, createdAt }, roomId, senderId, status, createdAt }	check if message is not deleted and return message
/chats/rooms	POST	{ name, members: [userId] }	{ room: { id, name, members [id, user: { id, name, image }] } }	create room
/chats/rooms?offset&limit	GET	-	{ room: { id, name, members [id, user: { id, name, image }] } }	return all the rooms in which the current user is a member, handle pagination parameters
/chats/rooms/:id	GET	-	{ room: { id, name, members [id, user: { id, name, image }] } }	return room
/chats/rooms/:id/messages	POST	{ text, attachments: { mimeType, file } }	{ id, text, attachments: { id, mimeType, originalName, url, createdAt }, senderId, status, createdAt, roomId }	Create message with matching room id
/chats/rooms/:id/messages?offset&limit	GET	-	{ id, text, attachments: { id, mimeType, originalName, url, createdAt }, senderId, status, createdAt, roomId }	return messages of the matching room id, handle pagination parameters
/chats/rooms/:id/members	POST	{ members: [userId] }	{ room: { id, name, members [id, user: { id, name, image }] } }	add members to the room
/chats/rooms/:id/members	GET	-	{ members [id, user: { id, name, image }] } }	return all the memebers with matching room id
/chats/rooms/:id/members/:memberId	GET	-	{ member [id, user: { id, name, image }] } }	return member by matching id
/chats/rooms/:id/members/:memberId	DELETE	-	-	remove member form room if he is the last member then delete the room

Chat Service Events

Channel	Data	Description
---------	------	-------------

Channel	Data	Description
---------	------	-------------

Channel	Data	Description
---------	------	-------------

No Events are emitted by this service

Messages Collection	
PK	<u>Id</u>
	text: String { required, can be null if there is an attachment } attachment: [id: ObjectId(), type: String ['image', 'video', 'document', 'file'], url] { nullable } roomId: ObjectId() { required } senderId: ObjectId() { required } status: String ['read', 'deleted', 'draft', 'sent'] { default: sent } createdDate: Date { default: Date.now }

Rooms Collection	
PK	<u>Id</u>
	name: String { required }

Rooms Members Collection	
PK	<u>Id</u>
FK	roomId: ObjectId() { required }
FK	userId: ObjectId () { requird }

Database Dependencies

Microservice Name	Table / Collection Name
Authentication	User

Status Field Description For Messages Collection:

- read: Indicates that the message has been read by the recipient.
- deleted: Indicates that the message has been deleted by the sender or recipient.
- draft: Indicates that the message is a draft and has not been sent yet.
- sent: Indicates that the message has been sent by the sender.

Routes Restriction Rules • Admin has access to ◦ /finanace/earnings/users/:id [Method: GET] ◦ /finanace/earnings/:id [Method: GET] ◦ /finanace/paymentMethods [Method: POST, GET] ◦ /finanace/paymentMethods/:id [Method: GET, DELETE] ◦ /finanace/payouts [Method: GET, DELETE] ◦ /finanace/payouts/:id [Method: GET, PATCH, DELETE] ◦ /finanace/transactions [Method: GET, DELETE] ◦ /finanace/transactions/:id [Method: GET, PATCH, DELETE] ◦ /finanace/transactions/types [Method: POST, GET] ◦ /finanace/transactions/types/:id [Method: GET, PATCH, DELETE] • Talent has access to ◦ /finanace/earnings/users/:id [Method: GET] ◦ /finanace/earnings/:id [Method: GET] ◦ /finanace/paymentMethods [Method: POST, GET] ◦ /finanace/paymentMethods/:id [Method: GET, DELETE] ◦ /finanace/withdraws [Method: POST, GET] • Client has access to ◦ /finanace/payments [Method: POST] ◦ /finanace/paymentMethods [Method: POST, GET] ◦ /finanace/paymentMethods/:id [Method: GET, DELETE]				
---	--	--	--	--

Finance Service End-Points				
Path	Method	Body?	Return	Goal?
/finanace/earnings/users/:id	GET	-	{ id, balance, pendingAmmount, userId, createdAt }	Return earnings made by the current user and if the user is admin return all the earnings
/finanace/earnings/:id	GET	-	{ id, balance, pendingAmmount, userId, createdAt }	Return earning by matching the id only the owner user or admin
/finanace/withdraws	POST	{ ammount, paymentMethodId }	{ id, ammount, userId, paymentMethodId, status, ceatedDate }	Check the amount should <= earnings.balance, and transfer the requested amount to user selected payment method
/finanace/withdraws	GET	-	{ id, ammount, userId, paymentMethodId, status, ceatedDate }	Return withdraws made by the current user and if the user is admin return all the withdraws
/finanace/paymentMethods	POST	{ phoneNo, CNIC }	{ id, phoneNo, accountTitle, CNIC, verified, active, createdAt }	Add payment method of the current user
/finanace/paymentMethods	GET	-	{ id, phoneNo, accountTitle, CNIC, verified, active, createdAt }	Return all the payment methods of the current user
/finanace/paymentMethods/:id	GET	-	{ id, phoneNo, accountTitle, CNIC, verified, active, createdAt }	Return the matching id payment method
/finanace/paymentMethods/:id	DELETE	-	-	Mark the matching id payment method as inactive
/finanace/payments	POST	{ type, refId, ammount, paymentMethodId }	{ type, refId, ammount, paymentMethodId, status }	Charge the user from the selected method
/finanace/payouts	GET	-	{ id, amount, status, transactionId, createdAt }	Return all the payouts only for admin
/finanace/payouts	DELETE	-	-	Delete all the payouts only for admin
/finanace/payouts/:id	GET	-	{ id, amount, status, transactionId, createdAt }	Return the matching id payout only for the admin

/finanace /payouts/:id	PATCH	{ ammount }	{ id, amount, status, transactionId, createdAt }	Update the matching id payout only for the admin
/finanace /payouts/:id	DELETE	-	-	Delete the matching id payout only for the admin
/finanace /transactions	GET	-	{ id, refId, type: { id, title }, paymentId, status, createdAt }	Return all the transactions only for admin
/finanace /transactions	DELETE	-	-	Delete all the transactions only for admin
/finanace /tansactions/:id	GET	-	{ id, refId, type: { id, title }, paymentId, status, createdAt }	Return the matching id transaction only for the admin
/finanace /transactions/:id	PATCH	{ status }	{ id, refId, type: { id, title }, paymentId, status, createdAt }	Update the matching id transaction only for the admin
/finanace /transactions/:id	DELETE	-	-	Delete the matching id transaction only for the admin
/finanace /transactions/types	POST	{ title }	{ id, title }	Create transaction type only for the admin
/finanace /transactions/types	GET	-	{ id, title }	Return all transaction type only for the admin
/finanace /transactions /types/:id	GET	-	{ id, title }	Return the matching id transaction type only for the admin
/finanace /transactions /types/:id	PATCH	{ title }	{ id, title }	Update the matching id transaction type only for the admin
/finanace /transactions /types/:id	DELETE	-	{ id, title }	Delete the matching id transaction type only for the admin

Finance Service Events

Channel	Data	Description
---------	------	-------------

No Events are emitted by this service

Earnings Collection	
PK	<u>Id</u>
	balance: Float { default: 0.00 } pendingAmmount: Float { default: 0.00 } userId: ObjectId () { required } createdDate: Date { default: Date.now }

Withdraws Collection	
PK	<u>Id</u>
	ammount: Float { default: 0.00 } userId: ObjectId () { required } paymentMethodId: ObjectId () { required } status: String ['pending', 'processing', 'completed', 'failed', 'cancelled'] { default: 'pending' } createdDate: Date { default: Date.now }

PaymentMethods Collection	
PK	<u>Id</u>
	phoneNo: String { required } accountTitle: String CNIC: String { required } verified: boolean active: boolean createDate: Date { default: Date.now }

Payments Collection	
PK	<u>Id</u>
	ammount: Float { required } fromUserId: ObjectId () { required } toUserId: ObjectId () { required } paymentMeethodIdFrom: ObjectId () { required } paymentMeethodIdTo: ObjectId () { required } createDate: Date { default: Date.now }

Transactions Collection	
PK	<u>Id</u>
	refId: ObjectId() { required } typeId: ObjectId () { required } paymentId: ObjectId() { required } status: String ['pending', 'completed', 'cancelled', 'disputed', 'failed'] { default: 'pending'} createDate: Date { default: Date.now }

Payouts Collection	
PK	<u>Id</u>
	ammount: Float { required } status: String [] { default } transactionId: ObjectId() { required } createDate: Date { default: Date.now }

TransactionTypes Collection	
PK	<u>Id</u>
	title: String { required }

Database Dependencies	
Microservice Name	Table / Collection Name
Authentication	User

Status Field Description For Withdraws Collection:

- pending: The withdrawal request has been submitted but has not yet been processed.
- processing: The withdrawal request is currently being processed.
- completed: The withdrawal request has been successfully processed and the funds have been transferred to the recipient's account.
- failed: The withdrawal request has failed due to an error or issue with the payment method.
- canceled: The withdrawal request has been canceled by the recipient or the platform.

Status Field Description For Transactions Collection:

- pending: The transaction has been initiated but not yet completed
- completed: The transaction has been successfully completed
- canceled: The transaction has been canceled by either the client or the freelancer
- disputed: The transaction has been disputed and is under review
- failed: The transaction failed to complete for some reason

Routes Restriction Rules

- Everyone has access to all routes

File Service End-Points

Path	Method	Body?	Return	Goal?
/files	POST	{ mimeType, file }	{ id, mimeType, originalName, url, createdAt }	Create the file
/files	GET	-	{ id, mimeType, originalName, url, createdAt }	Return all the files
/files/:id	GET	-	{ id, mimeType, originalName, url, createdAt }	Return the file with matching id
/files/:id	PATCH	{ mimeType, file }	{ id, mimeType, originalName, url, createdAt }	Update the file with matching id
/files/:id	DELETE	-	-	Delete the file with matching id

File Service Events

Channel	Data	Description
-		

File structure

While uploading and saving files following structure should be followed

- profiles/[images,videos]/filename.ext [hashed]
- projects/[images,videos]/filename.ext [hashed]
- products/[images,videos]/filename.ext [hashed]
- services/[images,videos]/filename.ext [hashed]
- jobs/[images,videos]/filename.ext [hashed]
- proposals/[images,videos]/filename.ext [hashed]

Files Collection

PK	<u>Id</u>
	contentType: String { required }
	originalName: String { required }
	url: String { required }
	userId: ObjectId() { required }
	createdAt: Date { default: Date.now }

Database Dependencies

Microservice Name

Table / Collection Name

Authentication

User